HATHOR-CORE · MASTER-BOOK

The Streaming Surface — WebSockets

How the node pushes live updates to clients — the WebSocket protocol, Autobahn over Twisted, and the admin and event-queue streaming surfaces.

SUBJECT	hathor-core · Part II · the node, end to end
CHAPTER	36 · Part II · The Node
BRANCH	study/hathor-core-studies
EDITION	2026 · v1

WebSocket · Autobahn · Twisted · Full-duplex · Server push · HTTP upgrade · Admin stream · Event-queue stream · JSON frames · Subscriptions

Table of Contents

Chapter 36 — The Streaming Surface: WebSockets

What you'll learn in this chapter

- What a **WebSocket** is, the request-response limitation it overcomes, and why a node needs to push data to clients rather than wait to be asked.
- How the **HTTP-upgrade handshake** turns an ordinary web request into a persistent, full-duplex channel — and why the node reuses its existing HTTP server to host it.
- How **Autobahn** layers a WebSocket implementation onto Twisted, and how `hathor-core` subclasses its factory and protocol classes.
- The **two distinct WebSocket surfaces** the node exposes — the admin stream (wallet/dashboard updates) and the event-queue stream (gap-free replay) — what each carries, and who consumes it.
- How clients **subscribe** to addresses, request history streams, and receive **JSON-framed** messages with rate limiting and flow control.
- Where the WebSocket factories are **built, mounted, and started** in the node's lifecycle.

Every chapter so far has been about the node talking to itself or to its peers: storage, verification, consensus, the P2P protocol. This chapter is about the node talking to **clients** — the wallets, dashboards, and downstream integrations that are not other full nodes but humans and applications wanting to watch the ledger as it changes. That requires a different kind of channel than the ones we have seen, and a different third-party library to build it. This is the canonical treatment of **WebSockets** and the **Autobahn** library in this book.

36.1 The problem: HTTP cannot push

You almost certainly know the shape of a normal web request. A client opens a connection, sends a request (`GET /balance/abc123`), the server sends a response, and — in the classic model — the conversation is over. The client asked; the server answered. This is the **request-response**[1] model, and it is the whole of plain HTTP.

That model has one property that matters enormously here: **the server can only speak when spoken to.** It has no way to start a sentence on its own. If a new block arrives and the server would like to tell an interested wallet "your balance just

changed," it cannot. It has no open channel to do so, and HTTP gives it no way to open one. The information flows in exactly one direction at a time, and only in reply to a request.

For a great many web pages this is fine — you click a link, you get a page. But a full node is a living system: blocks land, transactions get confirmed, balances shift, peers come and go, all continuously and unpredictably. A wallet UI wants to reflect those changes the instant they happen. A monitoring dashboard wants a live readout of the node's metrics. None of these clients knows when the next interesting thing will occur — so under pure HTTP they have only one option, and it is a bad one.

Polling, and why it is wasteful

The workaround under request-response is **polling**[2]: the client repeatedly asks "anything new yet? anything new yet?" on a timer — say every two seconds. Each poll is a fresh HTTP request: a new connection (or at least a new round-trip), headers sent and parsed, a handler invoked, a response built. Consider what this costs:

- **Latency.** If you poll every two seconds, a change can take up to two seconds to surface. Tighten the interval to feel responsive and you multiply the next cost.
- **Wasted work.** The overwhelming majority of polls return "nothing new." You pay the full price of a request-response round-trip to learn that nothing happened. Multiply by every connected client and every poll interval, and a node spends real CPU and bandwidth answering a question whose answer is almost always "no."
- **It scales the wrong way.** The busier you want the UI to feel, the faster you must poll, the more empty requests you generate. Responsiveness and efficiency pull in opposite directions.

Polling is the client simulating a live feed by asking very often. What we actually want is for the server to **push**[3]: to keep a channel open and send an update the moment one exists, with no request to trigger it. That is precisely what a WebSocket provides.

Full-duplex: both sides can talk at once

A telephone call is **full-duplex**[4]: both people can speak at the same time, and either can start talking whenever they like. A walkie-talkie is **half-duplex**: only one side transmits at a time, and you must release the button before the other can reply. Plain HTTP is closer to the walkie-talkie — one side speaks, then the other, strictly in turn, always begun by the client.

A **WebSocket** is the telephone call. Once established, it is a single, long-lived, **full-duplex** connection: the client can send messages to the server, the server can send messages to the client, **independently and at any time**, with no request needed to

unlock a response. The server can stay silent for an hour and then, the instant a block arrives, push a message — and the client receives it immediately, having asked for nothing. That is the capability the node's streaming surface is built on.

> **Recap — request-response vs. push.** Under HTTP the server answers questions; it cannot raise its hand. Polling fakes a live feed by asking constantly, paying for a round-trip each time to usually learn nothing changed. A WebSocket replaces all of that with one persistent, full-duplex pipe the server can write to whenever it has news.

36.2 What a WebSocket actually is — the upgrade handshake

Here is the part that surprises people: a WebSocket connection starts its life as an ordinary HTTP request. It does not run on some separate, exotic port with its own listener. It begins as a normal `GET`, on the same HTTP server, and is then **upgraded** into a WebSocket. This is deliberate — it lets WebSockets travel through the same ports, proxies, and firewalls that already pass web traffic, and it lets a server host both plain HTTP endpoints and WebSocket endpoints side by side.

The mechanism is the HTTP **upgrade handshake**. The client sends a `GET` carrying two special headers:

```
GET /v1a/ws HTTP/1.1
Host: node.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

`Upgrade: websocket` and `Connection: Upgrade` are the request: "I would like to stop speaking HTTP on this connection and start speaking the WebSocket protocol." The server, if it agrees, answers with a special status code — `101 Switching Protocols` — and a confirming header:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

From that `101` onward, the rules change. The same TCP[5] connection that carried the HTTP request stays open, but the two sides stop exchanging HTTP requests and responses and instead exchange WebSocket **frames**[6] — small, framed messages that can flow either direction at any time. The handshake is the hinge: HTTP on one side of it, full-duplex WebSocket on the other, over one unbroken socket.

The `Sec-WebSocket-Key` / `Sec-WebSocket-Accept` exchange is a small handshake-validation ritual (the server hashes the client's key with a fixed value and echoes it back) that proves to the client it is genuinely talking to a WebSocket-aware server and not a confused cache. You will not have to implement any of this by hand — the library does it — but it is worth knowing the `101` and the two `Upgrade` headers are the whole trick. **An HTTP request walks in; a WebSocket walks out.**

This is also why, in the code we are about to read, the WebSocket lives as a child of the HTTP resource tree. The node already runs an HTTP server (its REST API); the WebSocket endpoint is mounted on a path of that same server, and the upgrade happens when a client hits that path with the `Upgrade` headers.

36.3 Localization

The streaming surface is split across **two** packages, because the node exposes two genuinely different WebSocket services. Keep them apart in your mind from the start; §36.7 explains why they are separate.

```
hathor/
├── websocket/                  ← surface 1: the ADMIN / general stream
│   ├── __init__.py             ←   exports factory, protocol, stats resource
│   ├── factory.py              ←   HathorAdminWebsocketFactory  (the server)
│   ├── protocol.py             ←   HathorAdminWebsocketProtocol (one per client)
│   ├── resource.py             ←   WebsocketStatsResource (an HTTP stats endpoint)
│   ├── messages.py             ←   Pydantic message types (JSON frame shapes)
│   ├── streamer.py             ←   HistoryStreamer (push-producer for big history)
│   ├── iterators.py            ←   address-search helpers for history streaming
│   └── exception.py
│
└── event/
    └── websocket/              ← surface 2: the EVENT-QUEUE stream
        ├── __init__.py
        ├── factory.py          ←   EventWebsocketFactory  (the server)
        ├── protocol.py         ←   EventWebsocketProtocol (one per client)
        ├── request.py          ←   client→server request models
        └── response.py         ←   server→client response models
```

There is a third, smaller WebSocket factory worth naming so it does not confuse you later — `MiningWebsocketFactory` in `hathor/mining/ws.py`, mounted at `/mining_ws` — but it belongs to the mining surface and is covered in Chapter 37. This chapter is about the two general-purpose streams above.

Context. The `hathor/websocket/` package is the node's live link to wallets and dashboards — it pushes balance changes, new accepted transactions, and periodic metrics, and it can stream an address's whole history on demand. The `hathor/event/websocket/` package is the live link to downstream data systems — it replays the node's durable event log (Chapter 30) in strict order with no gaps. Both ride on the node's existing HTTP server via Autobahn, and both are driven by the one Twisted reactor that drives everything else.

36.4 The concepts it rests on

This chapter leans on three things taught in full elsewhere. We re-establish each just enough to read the code, then point you back.

Recap — Twisted, the reactor, factories and protocols (full treatment in Ch. 16). Twisted is the asynchronous-networking framework the whole node runs on. Its centre is the **reactor**: one event loop that waits for things to happen (data arriving, a timer firing) and calls your code in response, never blocking. In Twisted's model, a **Protocol** object represents one connection and holds its per-connection state, while a **Factory** is the long-lived object that manufactures a fresh protocol instance for each new connection and holds state shared across all of them. You will see exactly this split below: one factory per WebSocket service, one protocol instance per connected client. `LoopingCall` (a repeating timer) and `reactor.callLater` (a one-shot delayed call) also reappear here. → Ch. 16.

Recap — the event system this streams (full treatment in Ch. 30). Chapter 30 described two layers. **PubSub** is the node's in-process announcement bus: components `publish` events like `NETWORK_NEW_TX_ACCEPTED` or `WALLET_BALANCE_UPDATED`, and any component can `subscribe` to be called back when one fires. On top sits the **EventManager**, which (when the event queue is enabled) persists a durable, gap-free, sequentially-numbered log of public events to RocksDB. The admin WebSocket of this chapter is a **subscriber on the PubSub bus**; the event-queue WebSocket is a **reader of the EventManager's durable log**. → Ch. 30.

Recap — publish/subscribe (full treatment in Ch. 3). Pub-sub decouples the announcer from the listeners: a publisher fires an event without knowing or caring who is listening, and subscribers register interest by event type. The admin factory's `subscribe()` method (below) is a textbook subscriber registration — it tells PubSub "call me whenever any of these ten event types fire." → Ch. 3.

36.5 Autobahn: a WebSocket implementation for Twisted

A WebSocket is a protocol — a precise specification of the handshake, the frame format, the masking rules, ping/pong keep-alives, and close codes. You do not want to implement that by hand. **Autobahn**[7] is the library that implements it, and crucially it ships a Twisted-native binding. That last word is why `hathor-core` chose it: the node is already a Twisted program, so a WebSocket library that speaks Twisten's Factory/Protocol vocabulary slots straight in with no impedance mismatch.

What it is and the problem it solves

- **What it is.** Autobahn is a mature Python library implementing the WebSocket protocol (and the WAMP messaging protocol on top, which the node does not use). The relevant piece is `autobahn.twisted.websocket`.
- **The problem it solves.** It handles the entire WebSocket mechanism — the upgrade handshake, framing, masking, fragmentation, ping/pong, close handshakes — so the application only deals with whole messages. Without it the node would be parsing frame headers and computing `Sec-WebSocket-Accept` hashes by hand.
- **Its core abstractions.** Two classes mirror Twisted's own Factory/Protocol pair: `WebSocketServerFactory` and `WebSocketServerProtocol`. You subclass them. And one Twisted-web adapter: `WebSocketResource`, which makes a WebSocket factory mountable on an HTTP resource tree so the upgrade can happen on an existing HTTP path.

How you use it — a tiny echo server

Before the real code, the smallest possible Autobahn-on-Twisted server. It accepts WebSocket connections and echoes back whatever it receives — the "hello world" of WebSockets. Read it for the shape, not the details:

```
from autobahn.twisted.websocket import WebSocketServerFactory, WebSocketServerProtocol
from twisted.internet import reactor

class EchoProtocol(WebSocketServerProtocol):
    def onConnect(self, request):
        print("client connecting:", request.peer)   # handshake starting

    def onOpen(self):
        print("connection open")                    # handshake done, full-duplex now

    def onMessage(self, payload, isBinary):
        self.sendMessage(payload, isBinary)          # push it straight back

    def onClose(self, wasClean, code, reason):
        print("connection closed:", reason)

factory = WebSocketServerFactory()
factory.protocol = EchoProtocol                      # factory makes one EchoProtocol per client
reactor.listenTCP(9000, factory)
reactor.run()
```

Note the four lifecycle hooks Autobahn calls on the protocol — `onConnect` (handshake beginning), `onOpen` (handshake finished, channel live), `onMessage` (a frame arrived), `onClose` (connection ended) — and the one method you call to push data, `sendMessage`. Every real protocol in this chapter overrides exactly these hooks. The factory-makes-a-protocol-per-connection pattern is identical to Twisten's, because Autobahn's classes are Twisted Factory/Protocol subclasses. Hold this toy in mind; the real code is this skeleton with the bodies filled in.

36.6 Surface 1 — the admin WebSocket, walked

This is the wallet-and-dashboard stream. Its job: keep connected clients live-updated about wallet-relevant events and node metrics, and serve on-demand history streams.

The factory: one server, many clients

`HathorAdminWebsocketFactory` subclasses Autobahn's `WebSocketServerFactory` (`hathor/websocket/factory.py:77`). It binds the protocol class and tracks every live connection:

```
class HathorAdminWebsocketFactory(WebSocketServerFactory):
    protocol = HathorAdminWebsocketProtocol          # factory.py:81

    def buildProtocol(self, addr):                   # factory.py:86
        return self.protocol(self, is_history_streaming_enabled=self.is_history_streaming_enabled
```

`buildProtocol` is the Twisted hook that mints one protocol object per incoming connection — the toy above relied on the default; here it is overridden so each protocol receives a back-reference to the factory and a flag. In its `__init__` (`factory.py:89`) the factory holds the pieces of shared state that all connections need:

```
self.connections: set[HathorAdminWebsocketProtocol] = set()          # factory.py:101
self.address_connections: defaultdict[str, set[...]] = defaultdict(set)  # factory.py:107
self.rate_limiter = RateLimiter(reactor=self.reactor)                 # factory.py:111
self._lc_send_metrics = LoopingCall(self._send_metrics)              # factory.py:123
```

Why each of these exists is worth pausing on, because the factory's whole job is captured here:

- **`connections`** — the set of all clients that have finished handshaking. To broadcast (send the same message to everyone), the factory iterates this set. Note it holds only fully-open connections; a connection that drops mid-handshake never enters it.
- **`address_connections`** — a map from a wallet address to the set of clients that asked to watch that specific address. Some events are not broadcast to everyone — they are relevant only to whoever subscribed to the address involved. This map makes targeted delivery possible.
- **`rate_limiter`** — a guard against flooding a client. Under a burst of activity the node could try to push hundreds of messages a second; the limiter caps the rate per message type and buffers the overflow (more below).
- **`_lc_send_metrics`** — a `LoopingCall`, i.e. a repeating timer, that fires `_send_metrics` on a fixed interval to push a dashboard snapshot. This is server-initiated push in its purest form: no client asked, the timer fired, the data goes out.

Subscribing to the event bus

The factory is a PubSub subscriber. When the node builds it, `subscribe()` registers the factory's `handle_publish` callback for ten event types (`factory.py:166`):

```
def subscribe(self, pubsub):                              # factory.py:166
    events = [
        HathorEvents.NETWORK_NEW_TX_ACCEPTED,
        HathorEvents.WALLET_OUTPUT_RECEIVED,
        HathorEvents.WALLET_INPUT_SPENT,
        HathorEvents.WALLET_BALANCE_UPDATED,
        # ... ten in total
    ]
    for event in events:
        pubsub.subscribe(event, self.handle_publish)
```

From now on, whenever any of those events is published anywhere in the node, `handle_publish` runs (`factory.py:185`). It calls `serialize_message_data` to turn the event's Python arguments into a JSON-friendly `dict` (`factory.py:193` — e.g. for an accepted transaction it builds `tx.to_json_extended()`), stamps the message with a `type` field, and hands it to `send_or_enqueue`. **This is the push pathway end to end:** a block lands → consensus publishes `NETWORK_NEW_TX_ACCEPTED` on PubSub → the factory's callback fires → the data is serialized to JSON → it is sent to every connected client. No client polled for it.

Broadcast vs. targeted send

`send_message` decides the audience (`factory.py:252`):

```
def send_message(self, data):                          # factory.py:252
    if data['type'] in ADDRESS_EVENTS:                 # e.g. WALLET_ADDRESS_HISTORY
        if data['address'] in self.address_connections:
            self.execute_send(data, self.address_connections[data['address']])
    else:
        self.broadcast_message(data)
```

Address-scoped events (`ADDRESS_EVENTS`, `factory.py:70`) go only to the clients that subscribed to that address — that is the point of the `address_connections` map. Everything else is broadcast to all. The actual send is `execute_send` (`factory.py:228`): it serializes the dict to bytes with `json_dumpb`, then loops the target connections calling `c.sendMessage(payload, False)` — Autobahn's push primitive, the same `sendMessage` from the toy. The `False` means "this is a text frame, not binary." It swallows `Disconnected` (the client vanished) so one dead client cannot break the broadcast to the rest.

Rate limiting: protecting the client from a firehose

`send_or_enqueue` (`factory.py:262`) sits in front of `send_message` for the four high-volume event types listed in `CONTROLLED_TYPES` (`factory.py:42`). The logic: if a buffer is already pending, or if this hit would exceed the configured rate, the message is enqueued in a bounded `deque` instead of sent immediately (`enqueue_for_later`, `factory.py:282`), and a `reactor.callLater` is scheduled to drain the buffer later (`process_deque`, `factory.py:299`). The deque is bounded (`maxlen`), so under sustained overload the oldest throttled messages are silently dropped rather than letting memory grow without limit. Each enqueued message is tagged `throttled: True` so the client can tell it was delayed. This is back-pressure: when the producer outruns what a client should reasonably receive, the node degrades gracefully rather than drowning the client or itself.

The metrics timer

`_send_metrics` (`factory.py:153`) is the LoopingCall target. It assembles a snapshot — transaction and block counts, best-block height, hash rate, connected peers — tags it `type: 'dashboard:metrics'`, and broadcasts it. Started in `start()` (`factory.py:126`) with `self._lc_send_metrics.start(settings.WS_SEND_METRICS_INTERVAL, now=False)`, it is the node's heartbeat to every dashboard, on a timer, with no prompting.

The protocol: one client's state machine

`HathorAdminWebsocketProtocol` subclasses `WebSocketServerProtocol` (`hathor/websocket/protocol.py:41`). One instance exists per connected client, and it implements the same four Autobahn hooks as the toy:

```
def onOpen(self):                                       # protocol.py:94
    self.factory.on_client_open(self)                   # register me in the factory's set
    self.send_capabilities()                            # tell the client what I support

def onClose(self, wasClean, code, reason):              # protocol.py:100
    self.factory.on_client_close(self)                  # deregister, drop my address subs

def onMessage(self, payload, isBinary):                 # protocol.py:105
    message = json_loadb(payload)                       # parse the JSON frame
    _type = message.get('type')
    if _type == 'ping':             self._handle_ping(message)
    elif _type == 'subscribe_address':   self.factory._handle_subscribe_address(self, message)
    elif _type == 'request:history:xpub':  self._open_history_xpub_streamer(message)
    # ... dispatch on the message's "type" field
```

`onMessage` is a **dispatch table** keyed on the incoming JSON's `type` field (`protocol.py:120`) — the client-to-server half of the conversation. A client can:

- send `{"type": "ping"}` and get `{"type": "pong"}` back (a liveness check the client initiates);
- send `{"type": "subscribe_address", "address": "..."}` to start receiving events for one address — handled by `_handle_subscribe_address` on the factory (`factory.py:317`), which enforces per-connection subscription limits (`WS_MAX_SUBS_ADDRS_CONN`, `WS_MAX_SUBS_ADDRS_EMPTY`) and replies with a success/failure frame;
- request a **history stream** of an address's whole transaction history, by xpub[8] or by an explicit address list.

JSON message framing, typed

Every frame is JSON. The outbound message shapes are not hand-built dicts everywhere — for the streaming protocol they are Pydantic models in `hathor/websocket/messages.py`, each with a `Literal` `type` discriminator:

```
class CapabilitiesMessage(WebSocketMessage):            # messages.py:30
    type: Literal['capabilities'] = 'capabilities'
    capabilities: list[str]

class StreamVertexMessage(StreamBase):                  # messages.py:58
    type: Literal['stream:history:vertex'] = 'stream:history:vertex'
    id: str
    seq: int
    data: dict[str, Any]
```

Using Pydantic models (Chapter 18) for the frames means the message shape is validated and self-documenting, and `message.json_dumpb()` produces the bytes to push (`protocol.py:346`). The `type` field is the contract: the client switches on it exactly as the server switches on the inbound `type`.

The history streamer: pushing a large result safely

Streaming an address's entire history could be huge — far more than fits in one frame, and enough to either block the reactor or overwhelm a slow client if dumped at once. `HistoryStreamer` (`hathor/websocket/streamer.py:59`) solves both. It is a Twisted **push-producer** (`@implementer(IPushProducer)`, `streamer.py:58`) with its own small state machine (`StreamerState`, `streamer.py:37`: `NOT_STARTED → ACTIVE → PAUSED/CLOSING → CLOSED`) and **flow control** by sequence numbers and acknowledgements:

- It sends framed messages — `stream:history:begin`, then interleaved `stream:history:address` and `stream:history:vertex` frames, then `stream:history:end` — each carrying an incrementing `seq` number.
- The client periodically sends `{"type": "request:history:ack", "ack": N}`. The streamer tracks the last acked seq (`set_ack`, `streamer.py:131`) and a **sliding window**: it will not run more than `window_size` messages ahead of the last ack. If the client falls behind, the streamer pauses; when an ack arrives, it resumes (`resume_if_possible`, `streamer.py:164`).

This is the same windowed flow-control idea TCP uses, implemented at the application layer: the fast server is kept in step with the slow client by waiting for acknowledgements, so a client on a thin connection is never buried and the reactor is never monopolized by one greedy stream.

36.7 Surface 2 — the event-queue WebSocket

The second surface lives in `hathor/event/websocket/` and exists for a different consumer with a different need. Where the admin stream gives wallets a live, lossy-tolerant feed of current events, the event-queue stream gives downstream data systems a **durable, gap-free, exactly-ordered replay** of the node's entire public event history (Chapter 30). Think of an indexer or analytics pipeline that must process every event the node ever emitted, in order, surviving restarts and slow consumers. That demands guarantees the broadcast admin stream deliberately does not make.

`EventWebsocketFactory` (`hathor/event/websocket/factory.py:31`) and `EventWebsocketProtocol` (`hathor/event/websocket/protocol.py:34`) are again an Autobahn factory/protocol pair, but the contract is reversed: instead of the server pushing on a whim, the client drives consumption with explicit windowed requests. The client speaks three request types (`hathor/event/websocket/request.py`):

- `START_STREAM` (with `last_ack_event_id` and `window_size`) — "begin sending me events, starting after this ID, and never run more than `window_size` ahead of my acks."
- `ACK` (with `ack_event_id` and `window_size`) — "I've processed up to here; you may advance, and here's my current window."

- `STOP_STREAM`.

These are Pydantic models behind a discriminated union (`request.py:57`), parsed in `onMessage` (`protocol.py:93`) and dispatched by a `match` statement (`protocol.py:104`). The crucial guarantee is in `can_receive_event` (`protocol.py:58`): the factory will send an event to a connection **only if** the stream is active, the event is exactly the next expected one (`event_id == self.next_expected_event_id()`), and the number of unacknowledged events is below the window. Events are therefore delivered **strictly in order, one gap-free sequence, throttled by the client's acks** — no broadcast, no rate-limit-and-drop. Where the admin stream may discard throttled messages, the event stream may never skip one.

Delivery is recursive and reactor-friendly: `send_next_event_to_connection` (`factory.py:105`) sends one event, then schedules itself again via `self._reactor.callLater(0, ...)` (`factory.py:117`). The `callLater(0)` yields back to the reactor between sends so a long replay does not block the event loop — it streams the backlog one event per reactor turn until the client's window is full or the backlog is exhausted. Live events arrive via `broadcast_event` (`factory.py:84`), which the EventManager calls as new events are persisted (`hathor/event/event_manager.py:196` and `:374`).

> **Why two surfaces?** Different guarantees for different consumers. The **admin** stream optimizes for liveness for many lightweight UI clients: broadcast, best-effort, rate-limited, drop-on-overload. The **event-queue** stream optimizes for completeness for a few heavyweight data clients: per-client cursor, in-order, gap-free, ack-driven, never-drop. Trying to serve both needs from one mechanism would force one consumer to accept the wrong trade-off, so the node runs two.

36.8 Why WebSockets, and why Autobahn

The chapter's technology choices, against their alternatives.

WebSocket vs. HTTP polling. Covered in §36.1: polling pays a full round-trip per check to usually learn nothing, trades latency against waste, and scales backwards. A WebSocket replaces N empty requests with one open pipe that carries data only when there is data. For a continuously-changing system watched by live UIs, this is the difference between a responsive feed and a wasteful approximation of one.

WebSocket vs. Server-Sent Events (SSE). SSE[9] is a lighter standard for server-to-client push over a plain HTTP response that is held open. It is simpler and is enough when only the server needs to push and the client never sends anything back. But the node's streams are genuinely **bidirectional**: clients subscribe to addresses, request

history streams, send acks, adjust their flow-control window, ping. That two-way conversation is exactly what SSE cannot do and a WebSocket can. The interactivity is what rules SSE out.

Autobahn vs. rolling your own. The WebSocket protocol is fiddly — framing, masking, fragmentation, ping/pong, close codes. Hand-rolling it would be error-prone and pointless. The real choice is which library, and there the deciding factor is the rest of the stack: the node is a Twisted program, and Autobahn is the mature library with a first-class **Twisted-native binding** (`autobahn.twisted.websocket`). It speaks Twisted's Factory/Protocol vocabulary and plugs into Twisted's web resource tree via `WebSocketResource` (next section). An asyncio-based WebSocket library (e.g. `websockets`) would force an awkward bridge into the reactor; Autobahn needs none. The choice falls directly out of the Twisted decision made back in Chapter 16.

36.9 How it plugs into the lifecycle

The wiring lives in the composition root's resource builder (`hathor/builder/resources_builder.py`), the same place the REST API tree is assembled (Chapter 24). The connecting fact from §36.2 is visible here: the WebSocket is mounted as a child of the HTTP resource tree, using Autobahn's `WebSocketResource` adapter.

```
from autobahn.twisted.resource import WebSocketResource     # resources_builder.py:19

# admin websocket, mounted at /ws on the HTTP server
ws_factory = HathorAdminWebsocketFactory(manager=self.manager,
                                         metrics=self.manager.metrics,
                                         address_index=self.manager.tx_storage.indexes.addresses)
root.putChild(b'ws', WebSocketResource(ws_factory))          # resources_builder.py:323
ws_factory.subscribe(self.manager.pubsub)                    # :330  ← becomes a PubSub subscribe

# event-queue websocket, only if the event queue is enabled
if self._args.x_enable_event_queue or self._args.enable_event_queue:   # :333
    root.putChild(b'event_ws', WebSocketResource(self.event_ws_factory))  # :334

self.manager.websocket_factory = ws_factory                  # :348  ← manager will start it
```

Three things to read out of this:

1. **`WebSocketResource(ws_factory)`** is the adapter from §36.2 made concrete: it wraps the Autobahn factory in something Twisted-web can mount with `putChild`. A `GET /v1a/ws` carrying the `Upgrade` headers hits this child, and Autobahn performs the `101` handshake — the upgrade happens on the existing HTTP server. The admin stream is always mounted (when the status/HTTP server is enabled at all); the event-queue stream is mounted **only** when `--enable-event-queue` (or its experimental `--x-` form) is passed, because it requires the durable event log to be running.

2. **`ws_factory.subscribe(self.manager.pubsub)`** is where the admin factory joins the PubSub bus (§36.6) — from this line on, ledger events flow to connected clients.

3. **`self.manager.websocket_factory = ws_factory`** hands the factory to the manager so the manager can start it at the right moment. That moment is `HathorManager.start()` (`hathor/manager.py:321`), right after components initialize and just before metrics:

```
if self.websocket_factory:                # manager.py:321
    self.websocket_factory.start()         # starts the rate limiter + the metrics LoopingCall
```

Symmetrically, `stop()` halts it (`manager.py:362`). The event-queue factory is started instead by the EventManager (`hathor/event/event_manager.py:114`, `self._event_ws_factory.start(stream_id=...)`), since its lifecycle is bound to the durable event log it serves.

The end-to-end picture, then: at build time the factories are constructed and mounted on the HTTP tree; the admin one subscribes to PubSub; at `manager.start()` the admin factory's timers begin; a client connects to `/v1a/ws`, Autobahn upgrades the connection, and from there every relevant ledger event the node publishes is pushed to that client over the open full-duplex channel — all of it running on the one Twisted reactor (Chapter 16) that also drives storage, consensus, and the P2P layer.

Recap

Surface	Package	Server / client classes	What it streams	Delivery model	Audience
Admin / general	`hathor/websocket/`	`HathorAdminWebsocketFactory` / `HathorAdminWebsocketProtocol`	accepted txs, wallet balance/output/input events, periodic dashboard metrics, on-demand address history	broadcast or address-targeted; rate-limited, drops on overload	wallets, dashboards (many light clients)

Surface	Package	Server / client classes	What it streams	Delivery model	Audience
Event-queue	`hathor/event/websocket/`	`EventWebsocketFactory` / `EventWebsocketProtocol`	the durable, numbered public event log (Ch 30)	per-client cursor, strictly in-order, gap-free, ack-windowed, never drops	indexers, analytics (few heavy clients)
Mining	`hathor/mining/ws.py`	`MiningWebsocketFactory`	mining-related updates	—	miners → **Ch 37**

Both general surfaces are Autobahn factory/protocol pairs subclassed onto Twisted's reactor; both are mounted as children of the existing HTTP server via `WebSocketResource`, so a plain `GET` with `Upgrade` headers is promoted to a full-duplex WebSocket by the `101` handshake; both push JSON-framed messages the moment the node has something to say. The split between them is the chapter's central lesson: **liveness for UIs, completeness for data pipelines, are different problems, and the node solves each with a stream tuned to it.** Everything in this chapter has been about the node serving clients — the fifth full-node responsibility from Chapter 0. The next chapter turns to how the node produces the blocks those clients are watching: **Chapter 37 — Mining**, where we meet block templates, the CPU miner, and the canonical treatment of the Stratum protocol.

---

1. **Request-response** is the interaction model of plain HTTP: the client sends a request and the server sends back exactly one response, then the exchange is finished. The server cannot initiate; it can only reply. ↩
2. **Polling** is a client repeatedly asking the server "is there anything new?" on a timer, to approximate a live feed. Each poll is a full request even when the answer is "no," which makes it wasteful — most polls return nothing. ↩
3. **Server push** means the server sends data to the client without the client having requested that specific data, using a connection kept open for the purpose. It is the inverse of polling. ↩
4. **Full-duplex** describes a channel on which both ends can transmit at the same time and either can start at any moment (like a phone call). **Half-duplex** allows only one direction at a time (like a walkie-talkie). Plain HTTP is effectively half-duplex and always client-initiated; a WebSocket is full-duplex. ↩
5. **TCP** (Transmission Control Protocol) is the reliable, ordered byte-stream transport that both HTTP and WebSockets run on top of. A WebSocket reuses the very TCP connection that carried the initial HTTP request — the upgrade does not open a new socket. ↩

6. A **frame** is the unit of data in the WebSocket protocol: a small message with a tiny header indicating its type (text, binary, ping, pong, close) and length. After the handshake, the two sides exchange frames instead of HTTP requests/responses. ↩
7. **Autobahn** is a Python library implementing the WebSocket protocol (and the higher-level WAMP protocol, unused here). `hathor-core` uses its Twisted binding, `autobahn.twisted.websocket`, whose `WebSocketServerFactory` / `WebSocketServerProtocol` mirror Twisted's own Factory/Protocol pair. ↩
8. An **xpub** (extended public key) is a single public key from which a whole sequence of wallet addresses can be derived without exposing any private key. The history streamer can walk an xpub's derived addresses to stream a wallet's entire history. ↩
9. **Server-Sent Events (SSE)** is a simpler push standard where the server holds an HTTP response open and writes events to it over time. It is one-directional (server→client only), which is why it cannot serve a stream where the client must also subscribe, ack, and adjust flow control. ↩